

# BÀI 1: CƠ SỞ LẬP TRÌNH BẢO MẬT, KIỂM TRA ĐẦU VÀO

Bài học này sẽ giới thiệu nền tảng cơ bản về lập trình bảo mật, tập trung vào việc kiểm tra đầu vào – một trong những khía cạnh then chốt để phòng chống lỗ hổng bảo mật. Sinh viên sẽ được làm quen với các khái niệm kiểm tra và xử lý dữ liệu đầu vào, hiểu tầm quan trọng của việc xác thực, lọc dữ liệu và phát hiện lỗi ngay từ bước đầu. Bài học cũng hướng dẫn thực hành sử dụng các công cụ hỗ trợ kiểm tra đầu vào và các kỹ thuật lập trình an toàn, tạo nền tảng vững chắc để áp dụng trong các bài bảo mật nâng cao sau này.

## **1.1 CƠ SỞ LẬP TRÌNH BẢO MẬT**

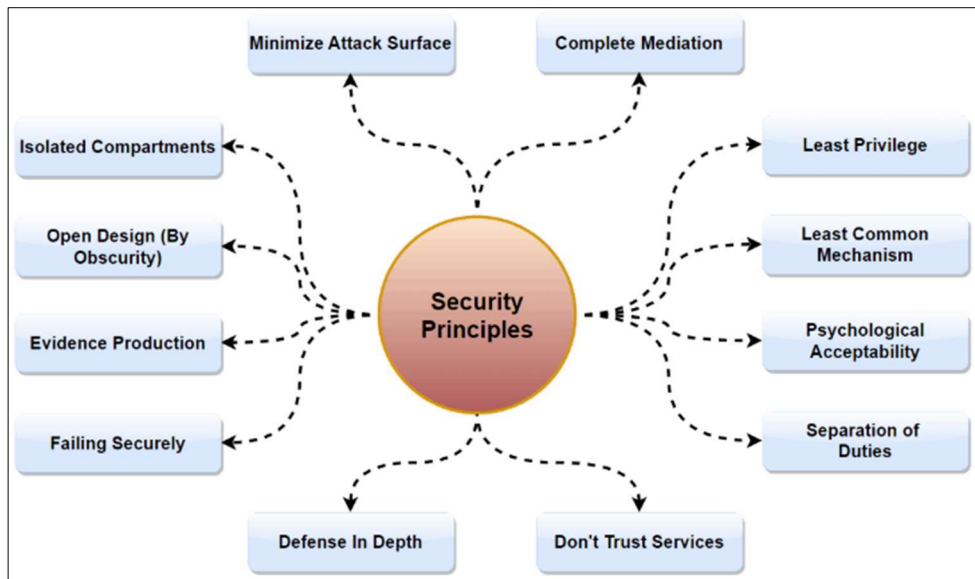
---

### **1.1.1 Nguyên tắc Bảo mật theo thiết kế**

**Bảo mật theo Thiết kế (Secure by Design)** là một trong những nền tảng quan trọng nhất trong phát triển phần mềm hiện đại. Ý tưởng cốt lõi của nguyên tắc này là bảo mật không phải là yếu tố được thêm vào sau cùng mà phải được tích hợp ngay từ giai đoạn thiết kế hệ thống. Điều này đồng nghĩa với việc các kiến trúc, quy trình, và các thành phần phần mềm cần được xây dựng sao cho giảm thiểu tối đa các lỗ hổng bảo mật tiềm ẩn. Cụ thể, việc áp dụng nguyên tắc Secure by Design bao gồm:

- Thiết kế hệ thống với các biện pháp kiểm soát quyền truy cập chặt chẽ, phân quyền rõ ràng và giới hạn tối đa quyền hạn cho các thành phần.
- Ưu tiên sử dụng các cơ chế mã hóa và bảo vệ dữ liệu nhạy cảm từ đầu.
- Đảm bảo tính toàn vẹn và khả năng phát hiện sai lệch của dữ liệu và các hoạt động trong hệ thống.
- Áp dụng nguyên tắc “ít đặc quyền nhất” (least privilege) cho mọi thành phần và người dùng trong hệ thống.
- Thiết kế hệ thống có khả năng chịu lỗi và phòng chống các cuộc tấn công phổ biến như injection, cross-site scripting, v.v.

Việc tuân thủ nguyên tắc bảo mật theo thiết kế giúp tăng cường đáng kể khả năng phòng thủ chủ động, giảm thiểu rủi ro bị khai thác và tạo nền tảng vững chắc cho các bước phát triển và vận hành tiếp theo của phần mềm.



**Hình 1.1: Các nguyên tắc bảo mật. Nguồn: Amadeus Labs.**

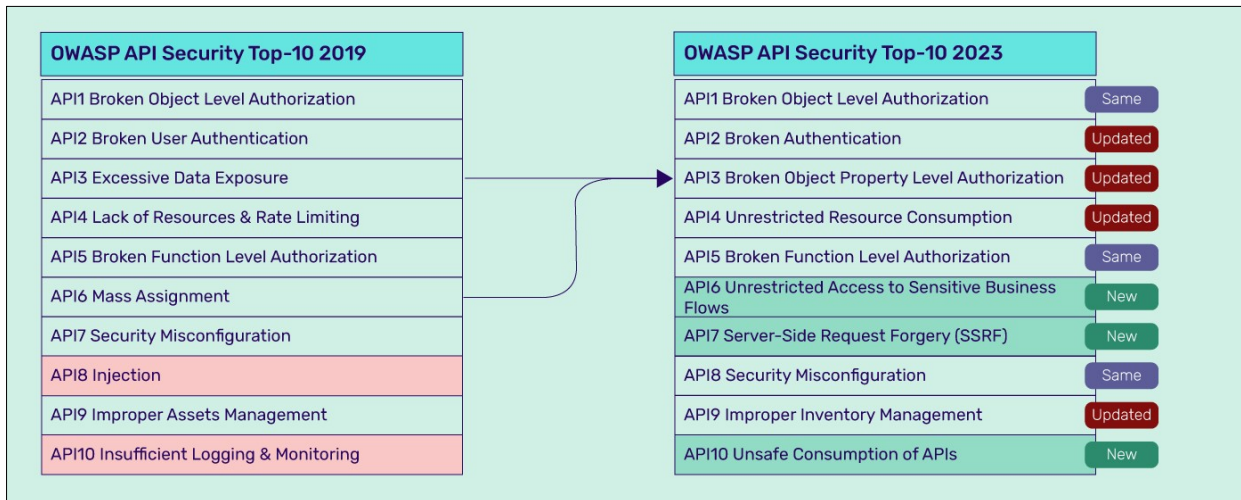
### 1.1.2 Thực hành lập trình an toàn theo OWASP

**OWASP (Open Web Application Security Project)** là tổ chức uy tín hàng đầu thế giới về bảo mật ứng dụng web. Các thực hành lập trình an toàn theo OWASP cung cấp bộ nguyên tắc, hướng dẫn cụ thể nhằm giúp lập trình viên phát triển phần mềm không chỉ hoạt động đúng chức năng mà còn bảo vệ hiệu quả trước các nguy cơ tấn công phổ biến. Thực hành lập trình an toàn theo OWASP bao gồm các điểm chính:

- Hiểu rõ và phòng chống các lỗ hổng phổ biến như Injection, Broken Authentication, Cross-Site Scripting (XSS), Security Misconfiguration, v.v., dựa trên danh sách OWASP Top 10.
- Áp dụng nguyên tắc "Defense in Depth" – xây dựng nhiều lớp phòng vệ để hạn chế rủi ro khi một lớp bảo mật bị vượt qua.
- Kiểm tra và xác thực đầu vào (Input Validation) nghiêm ngặt để ngăn chặn dữ liệu độc hại gây ảnh hưởng đến hệ thống.
- Sử dụng các thư viện và framework bảo mật được cập nhật thường xuyên, tránh tái phát triển các chức năng bảo mật quan trọng từ đầu.
- Quản lý phiên làm việc (session) và xác thực (authentication) an toàn để ngăn chặn đánh cắp, giả mạo thông tin người dùng.

- Ghi nhật ký và xử lý lỗi hợp lý, tránh để lộ thông tin nhạy cảm qua các thông báo lỗi.

Thực hành theo các hướng dẫn của OWASP không chỉ nâng cao chất lượng bảo mật của phần mềm mà còn giúp doanh nghiệp giảm thiểu rủi ro về tài chính và uy tín khi bị tấn công.



**Hình 1.2: OWASP API Top 10 2023. Nguồn: Indusface Blog.**

### 1.1.3 Nguyên tắc kiểm tra và làm sạch dữ liệu đầu vào

Dữ liệu đầu vào luôn là cửa ngõ nguy hiểm nhất mà hacker tìm cách khai thác để tấn công hệ thống. Vì vậy, việc **kiểm tra và làm sạch dữ liệu đầu vào** không thể coi nhẹ mà phải được thực hiện nghiêm túc theo những nguyên tắc cơ bản sau:

- Xác thực dữ liệu (**Validation**): Đảm bảo dữ liệu nhận được đúng định dạng, kiểu dữ liệu, độ dài, và phạm vi hợp lệ trước khi xử lý hoặc lưu trữ. Ví dụ, số điện thoại chỉ chứa chữ số, email đúng cú pháp chuẩn, ngày tháng hợp lệ.
- Làm sạch dữ liệu (**Sanitization**): Loại bỏ hoặc mã hóa các ký tự nguy hiểm có thể gây lỗi hoặc tạo điều kiện cho tấn công, như các ký tự đặc biệt trong câu lệnh SQL, mã HTML/JavaScript để ngăn chặn Injection và XSS.
- Nguyên tắc "Whitelist" ưu tiên hơn "Blacklist": Chỉ cho phép các giá trị, ký tự, hoặc mẫu dữ liệu được định nghĩa rõ ràng và an toàn, thay vì cố gắng lọc bỏ những thứ không mong muốn một cách ngẫu nhiên.
- Kiểm tra trên nhiều lớp: Validation và sanitization nên được thực hiện cả ở phía client (giao diện người dùng) để tăng trải nghiệm và giảm tải server, cũng như ở phía server để đảm bảo tính chắc chắn và an toàn tuyệt đối.

- Tránh tin tưởng dữ liệu từ bên ngoài: Mọi dữ liệu nhập từ người dùng, API hoặc bên ngoài đều phải được xử lý nghiêm ngặt trước khi đưa vào hệ thống.

Tuân thủ nguyên tắc kiểm tra và làm sạch dữ liệu đầu vào là bước quan trọng giúp bảo vệ ứng dụng khỏi các cuộc tấn công phổ biến như SQL Injection, Cross-Site Scripting, Command Injection, đồng thời đảm bảo hệ thống vận hành ổn định, không bị lỗi do dữ liệu không hợp lệ.

### **1.1.4 Xử lý lỗi an toàn và bảo mật ghi nhật ký**

Trong bất kỳ hệ thống phần mềm nào, việc xử lý lỗi và ghi nhật ký (logging) đóng vai trò then chốt trong bảo mật và duy trì ổn định hoạt động. Tuy nhiên, nếu không được thực hiện đúng cách, chúng có thể trở thành "cánh cửa mở" cho kẻ tấn công khai thác lỗ hổng.

#### **❖ Xử lý lỗi an toàn (Secure Error Handling):**

- Không tiết lộ thông tin nhạy cảm trong thông báo lỗi gửi về người dùng cuối. Ví dụ, không để lộ chi tiết lỗi cơ sở dữ liệu, cấu trúc hệ thống, hay đoạn mã nguồn.
- Cung cấp thông báo lỗi mang tính tổng quát, dễ hiểu nhưng không tiết lộ dữ liệu kỹ thuật.
- Ghi nhận lỗi chi tiết trong hệ thống nội bộ để phục vụ quá trình phân tích và xử lý.

#### **❖ Bảo mật ghi nhật ký (Secure Logging):**

- Nhật ký phải ghi lại đầy đủ các sự kiện quan trọng như lỗi hệ thống, truy cập, hành động người dùng, cảnh báo bảo mật.
- Dữ liệu ghi nhật ký cần được bảo vệ khỏi truy cập trái phép, đảm bảo tính toàn vẹn và khả năng truy xuất.
- Tránh ghi thông tin nhạy cảm như mật khẩu, dữ liệu cá nhân, hoặc thông tin định danh trong nhật ký. Nếu cần, phải mã hóa hoặc ẩn đi.
- Thiết lập cơ chế quản lý nhật ký như phân quyền truy cập, lưu trữ an toàn và định kỳ kiểm tra nhật ký để phát hiện sớm dấu hiệu tấn công.

Xử lý lỗi đúng cách và bảo mật ghi nhật ký không chỉ giúp tăng cường an toàn cho ứng dụng mà còn tạo điều kiện cho việc phát hiện, phản ứng kịp thời với các sự cố bảo mật, từ đó giảm thiểu thiệt hại và nâng cao uy tín hệ thống.

## 1.2 THỰC HÀNH: THƯ VIỆN XÁC THỰC ĐẦU VÀO

---

### 1.2.1 Yêu cầu

**Mục tiêu:** Xây dựng một thư viện bằng ngôn ngữ Python kiểm tra và làm sạch dữ liệu đầu vào (sau đây gọi là thư viện "SecureValidator"), đảm bảo an toàn cho ứng dụng trước các lỗ hổng bảo mật phổ biến.

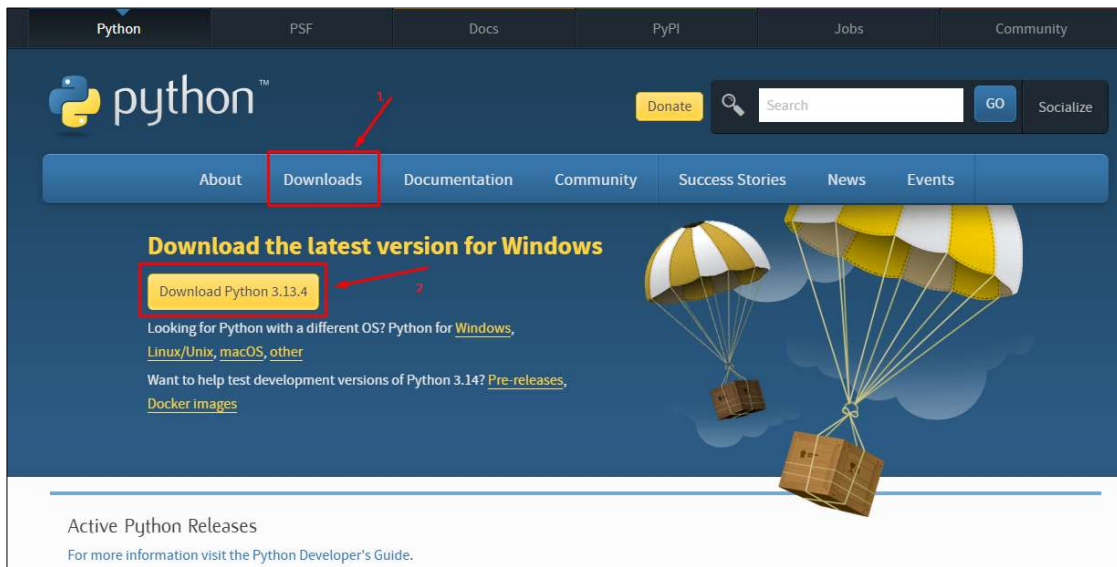
**Yêu cầu bài tập:**

- Cài đặt các hàm sau trong thư viện SecureValidator:
  - `validate_email(email)`: Kiểm tra định dạng email hợp lệ và ngăn ngừa tấn công chèn mã (injection).
  - `validate_url(url)`: Kiểm tra tính hợp lệ của URL và phòng chống tấn công SSRF (Server-Side Request Forgery).
  - `validate_filename(filename)`: Ngăn chặn các cuộc tấn công truy cập vượt thư mục (path traversal).
  - `sanitize_sql_input(input_str)`: Làm sạch dữ liệu đầu vào để phòng chống tấn công SQL Injection.
  - `sanitize_html_input(html_str)`: Làm sạch dữ liệu đầu vào để ngăn chặn tấn công XSS (Cross-Site Scripting).
- Viết bộ test case kiểm thử cho từng hàm, bao gồm cả dữ liệu hợp lệ và dữ liệu độc hại (malicious inputs).
- Trình bày minh họa kết quả xử lý của thư viện với các trường hợp đầu vào khác nhau.

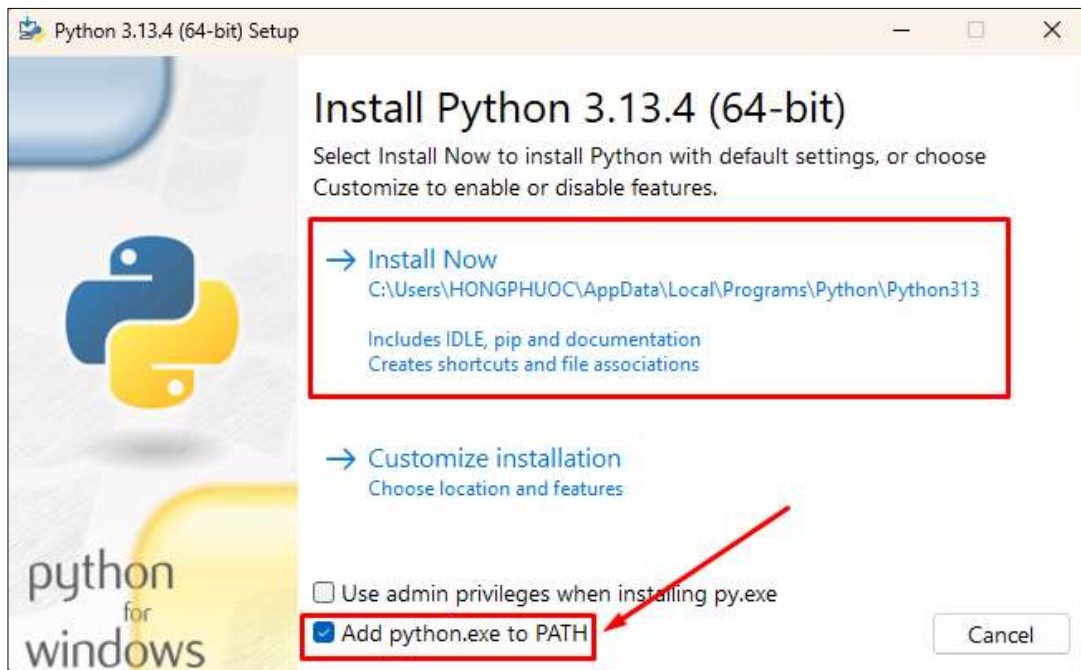
### 1.2.2 Cài đặt môi trường

#### 1. Cài đặt Python:

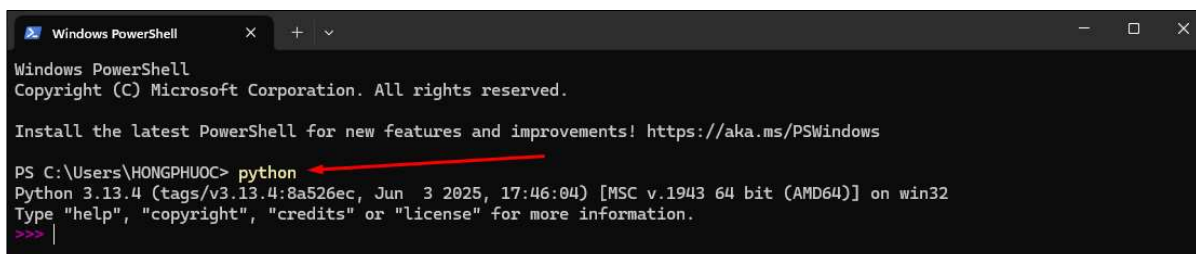
- Vào trang <https://www.python.org/downloads/> Chọn thẻ "Downloads" > Tải về phiên bản Python phù hợp.



- Sau khi tải file cài đặt về, tiến hành cài đặt Python vào máy tính.
- Lưu ý: Nhớ chọn vào tùy chọn "Add python.exe to PATH".



- Kiểm tra Python đã cài đặt> Vào Command Prompt và gõ lệnh: `python`



## 2. Tạo tài khoản Github và tạo mới Repository:

- Vào trang <https://github.com/> Đăng nhập nếu đã có tài khoản.
- Nếu chưa có tài khoản, chọn chức năng "Sign up" để đăng ký.
- Sau khi đăng nhập thành công > Chọn "New" > "Create a new repository".
- Nhập các thông tin cần thiết:
  - Repository name: TH\_LTANTT\_MSSV.
  - Description: Thuc hanh Lap trinh An ninh thong tin – MSSV – Ho va Ten.
  - Chọn chế độ "Private". Tích chọn vào "Add a README file".
  - Cuối cùng chọn "Create repository".

## Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (\*).

Owner \*

phucntmh

Repository name \*

TH\_LTANTT\_1511060249

TH\_LTANTT\_1511060249 is available.

Great repository names are short and memorable. Need inspiration? How about [legendary-octo-spoon](#) ?

Description (optional)

Thuc hanh Lap trinh An ninh thong tin – 1511060249 - Phuoc Nguyen

☐ Public

Anyone on the internet can see this repository. You choose who can commit.

☒ Private

You choose who can see and commit to this repository.

Initialize this repository with:

☒ Add a README file

This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: None

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: None

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

This will set `main` as the default branch. Change the default name in your [settings](#).

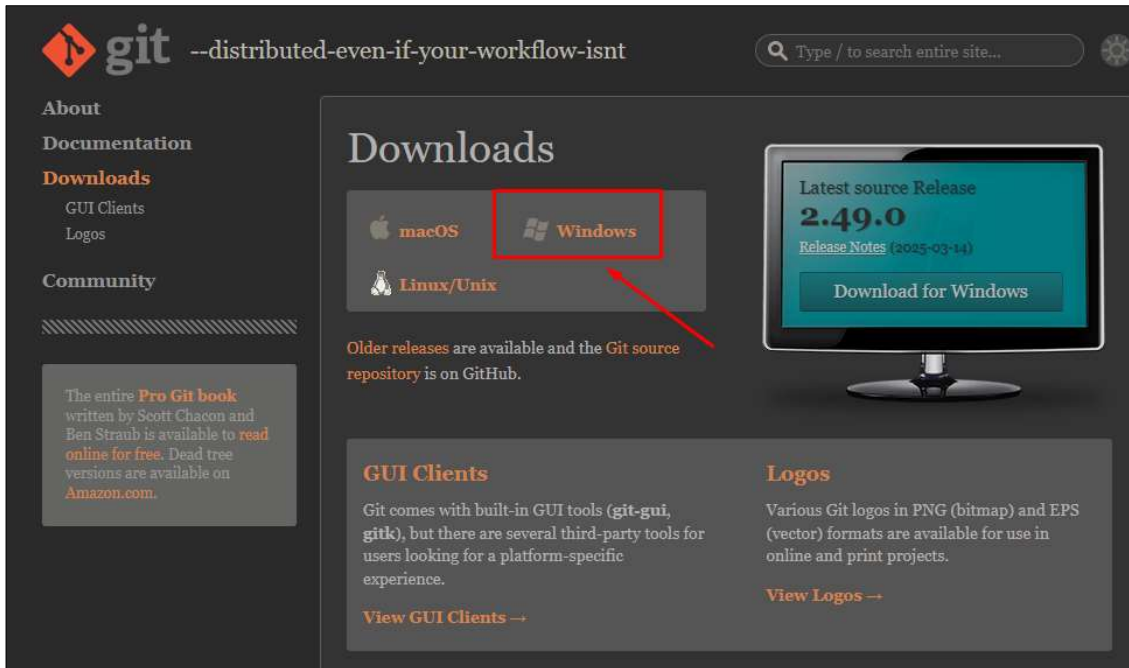
You are creating a private repository in your personal account.

Create repository

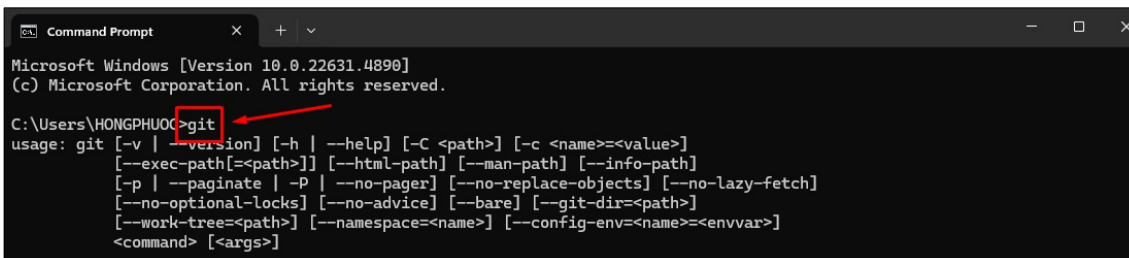
- Sau khi tạo xong, copy đường dẫn tới repo và lưu lại.

### 3. Cài đặt Git và clone repo về:

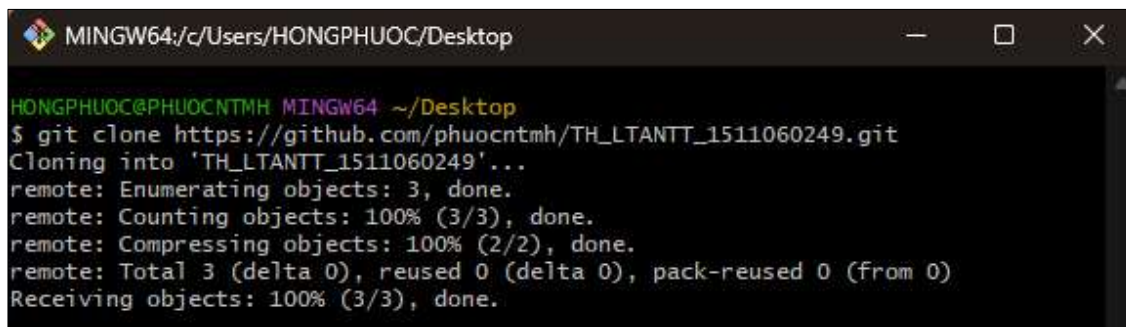
- Vào trang <https://git-scm.com/downloads> Tải về file cài đặt.
- Sau khi tải về, tiến hành cài đặt vào máy tính.



- Sau khi cài đặt, vào Command Prompt để kiểm tra bằng lệnh: `git`



- Mở cửa sổ Git bash để tiến hành clone repo từ Github về:

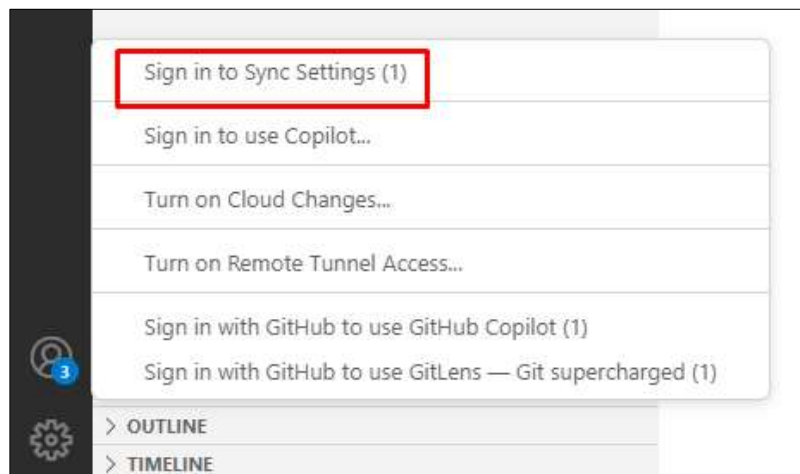


### 4. Cài đặt Visual Studio Code:

- Vào trang <https://code.visualstudio.com/download> và tải về phiên bản phù hợp với máy tính.
- Sau khi tải về, tiến hành cài đặt Visual Studio Code vào máy tính.



- Sau khi cài đặt xong, mở VS Code > Đăng nhập bằng tài khoản Github.



- Vào tab Extensions và cài đặt các extension cần thiết: Python Extension Pack, Pylance, Python Debugger, GitLens, Prettier.
- Tại tab Explorer > Chọn "Open Folder" > Mở folder đã clone về từ repo Github ở bước trước.
- Chọn "Yes, I trust the authors" khi được hỏi.

## 1.2.3 Hướng dẫn thực hành

### 1. Tạo thư viện SecureValidator:

- Tạo thư mục "secure-validator-lab". Trong thư mục này, tạo các thư mục và tệp tin theo cấu trúc sau:

```
secure-validator-lab/  
├─ app.py  
├─ requirements.txt  
├─ securevalidator/  
│   ├── __init__.py  
│   └─ core.py  
├─ templates/index.html  
└─ tests/test_validators.py
```

- Viết code thư viện securevalidator/core.py:

```
secure-validator-lab > securevalidator > core.py > ...  
1  import re, html, urllib.parse, os  
2  
3  def validate_email(email: str) -> bool:  
4      """Validate email format."""  
5      pattern = r'^[\w\.-]+@[\w\.-]+\.\w+$'  
6      return re.fullmatch(pattern, email) is not None  
7  
8  def validate_url(url: str) -> bool:  
9      """Validate URL and prevent basic SSRF vectors."""  
10     try:  
11         parsed = urllib.parse.urlparse(url)  
12         return parsed.scheme in ['http', 'https'] and bool(parsed.netloc)  
13     except Exception:  
14         return False  
15  
16  def validate_filename(filename: str) -> bool:  
17      """Prevent path traversal attacks."""  
18      if ".." in filename or "/" in filename or "\\" in filename:  
19         return False  
20      return os.path.basename(filename) == filename  
21  
22  def sanitize_sql_input(input_str: str) -> str:  
23      """Sanitize SQL input to prevent SQL injection."""  
24      sanitized = re.sub(r"(--|;|'|\"|#)", "", input_str)  
25      sanitized = re.sub(r"\b(OR|AND|SELECT|INSERT|DELETE|UPDATE|DROP|UNION|WHERE)\b",  
26         "", sanitized, flags=re.IGNORECASE)  
27      return sanitized.strip()  
28  
29  def sanitize_html_input(html_str: str) -> str:  
30      """Escape HTML input to prevent XSS."""  
31      return html.escape(html_str)
```

- Viết code thư viện securevalidator/\_\_init\_\_.py:

```
secure-validator-lab > securevalidator >  __init__.py
1 from .core import (
2     validate_email, validate_url, validate_filename,
3     sanitize_sql_input, sanitize_html_input
4 )
5
```

- Các gói cần thiết trong requirements.txt:

```
secure-validator-lab >  requirements.txt
1 Flask==2.3.3
2 gunicorn==21.2.0
3
```

- Viết code giao diện web templates/index.html:

```
secure-validator-lab > templates >  index.html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>SecureValidator</title>
6     <meta name="viewport" content="width=device-width, initial-scale=1.0">
7     <link rel="stylesheet" href="https://unpkg.com/@picocss/pico@1.*/css/pico.min.css">
8     <style>
9         body {margin:auto; max-width:720px;}
10        h1 {text-align:center; margin-bottom:2rem;}
11        .result-line {font-size:0.9rem; margin: -0.5rem 0 1rem; color:#333;}
12        .valid {color:green;}
13        .invalid {color:red;}
14        code {background:#f3f3f3; padding:0.2rem 0.4rem;
15            border-radius:0.3rem; font-size:0.95em;}
16    </style>
17 </head>
18 <body>
19     <main class="container">
20         <h1>SecureValidator</h1>
21         <form method="POST">
22             <label>Email:
23                 <input type="text" name="email" placeholder="you@example.com"
24                     value="{{ request.form.get('email', '') }}">
25                 {% if results is not none %}
26                     <div class="result-line {{ 'valid' if results.email else 'invalid' }}">
27                         {{ "Email hợp lệ" if results.email else "Email không hợp lệ" }}
28                     </div>
29                 {% endif %}
30             </label>
31
```

```

32 <label>URL:
33   <input type="text" name="url" placeholder="https://example.com"
34     value="{{ request.form.get('url', '') }}">
35   {% if results is not none %}
36     <div class="result-line {{ 'valid' if results.url else 'invalid' }}">
37       {{ "URL hợp lệ" if results.url else "URL không hợp lệ" }}
38     </div>
39   {% endif %}
40 </label>
41
42 <label>Filename:
43   <input type="text" name="filename" placeholder="file.txt"
44     value="{{ request.form.get('filename', '') }}">
45   {% if results is not none %}
46     <div class="result-line {{ 'valid' if results.filename else 'invalid' }}">
47       {{ "Tên file hợp lệ" if results.filename else "Tên file không hợp lệ" }}
48     </div>
49   {% endif %}
50 </label>
51
52 <label>SQL Input:
53   <input type="text" name="sql" placeholder="input SQL..."
54     value="{{ request.form.get('sql', '') }}">
55   {% if results is not none %}
56     <div class="result-line"><strong>Đã lọc:</strong>
57       <code>{{ results.sql }}</code></div>
58   {% endif %}
59 </label>

```

```

60
61 <label>HTML Input:
62   <input type="text" name="html" placeholder='&lt;script&gt;alert(1)&lt;/script&gt;'
63     value="{{ request.form.get('html', '') }}">
64   {% if results is not none %}
65     <div class="result-line"><strong>Đã mã hóa:</strong>
66       <code>{{ results.html }}</code></div>
67   {% endif %}
68 </label>
69
70   <button type="submit">Xác thực ngay</button>
71 </form>
72 </main>
73 </body>
74 </html>
75

```

- Viết code xử lý app.py:

```

secure-validator-lab > app.py > ...
1  from flask import Flask, render_template, request
2  from securevalidator import (
3      validate_email, validate_url, validate_filename,
4      sanitize_sql_input, sanitize_html_input
5  )
6
7  app = Flask(__name__)
8
9  @app.route("/", methods=["GET", "POST"])
10 def index():
11     results = None
12     if request.method == "POST":
13         results = {
14             "email": validate_email(request.form["email"]),
15             "url": validate_url(request.form["url"]),
16             "filename": validate_filename(request.form["filename"]),
17             "sql": sanitize_sql_input(request.form["sql"]),
18             "html": sanitize_html_input(request.form["html"]),
19         }
20     return render_template("index.html", results=results)
21
22 if __name__ == "__main__":
23     app.run(debug=True)

```

## 2. Tạo test cases cho Unit Test:

- Viết code cho file tests/test\_validators.py:

```

secure-validator-lab > tests > test_validators.py > TestValidators > setUp
1  import unittest
2  from securevalidator import (
3      validate_email, validate_url, validate_filename,
4      sanitize_sql_input, sanitize_html_input
5  )
6
7  class TestValidators(unittest.TestCase):
8      def setUp(self):
9          print("\n Running:", self._testMethodName)
10
11      def test_validate_email_valid(self):
12          self.assertTrue(validate_email("user@example.com"))
13
14      def test_validate_email_invalid(self):
15          self.assertFalse(validate_email("user@@example..com"))
16

```

```

17 def test_validate_url_valid(self):
18     self.assertTrue(validate_url("https://example.com"))
19
20 def test_validate_url_invalid(self):
21     self.assertFalse(validate_url("ftp://example.com"))
22
23 def test_validate_filename_valid(self):
24     self.assertTrue(validate_filename("report.pdf"))
25
26 def test_validate_filename_traversal(self):
27     self.assertFalse(validate_filename("../../etc/passwd"))
28
29 def test_sanitize_sql_input_injection(self):
30     input_str = "' OR 1=1 --"
31     sanitized = sanitize_sql_input(input_str)
32     self.assertNotIn("'", sanitized)
33     self.assertNotIn("--", sanitized)
34     self.assertNotIn("OR", sanitized.upper())
35
36 def test_sanitize_sql_input_safe_text(self):
37     input_str = "hello world"
38     sanitized = sanitize_sql_input(input_str)
39     self.assertEqual(sanitized, "hello world")
40
41 def test_sanitize_html_input_script(self):
42     input_str = '<script>alert("XSS")</script>'
43     sanitized = sanitize_html_input(input_str)
44     self.assertEqual(sanitized, '&lt;script&gt;alert(&quot;XSS&quot;)&lt;/script&gt;')
45
46 def test_sanitize_html_input_safe_text(self):
47     input_str = "Hello World"
48     sanitized = sanitize_html_input(input_str)
49

```

- Vào Terminal > Chạy lệnh: `python -m unittest discover tests`

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> cd .\secure-validator-lab\
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\secure-validator-lab> python -m unittest discover tests

Running: test_sanitize_html_input_safe_text
.
Running: test_sanitize_html_input_script
.
Running: test_sanitize_sql_input_injection
.
Running: test_sanitize_sql_input_safe_text
.
Running: test_validate_email_invalid
.
Running: test_validate_email_valid
.
Running: test_validate_filename_traversal
.
Running: test_validate_filename_valid
.
Running: test_validate_url_invalid
.
Running: test_validate_url_valid
.
-----
Ran 10 tests in 0.003s

OK
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\secure-validator-lab>

```

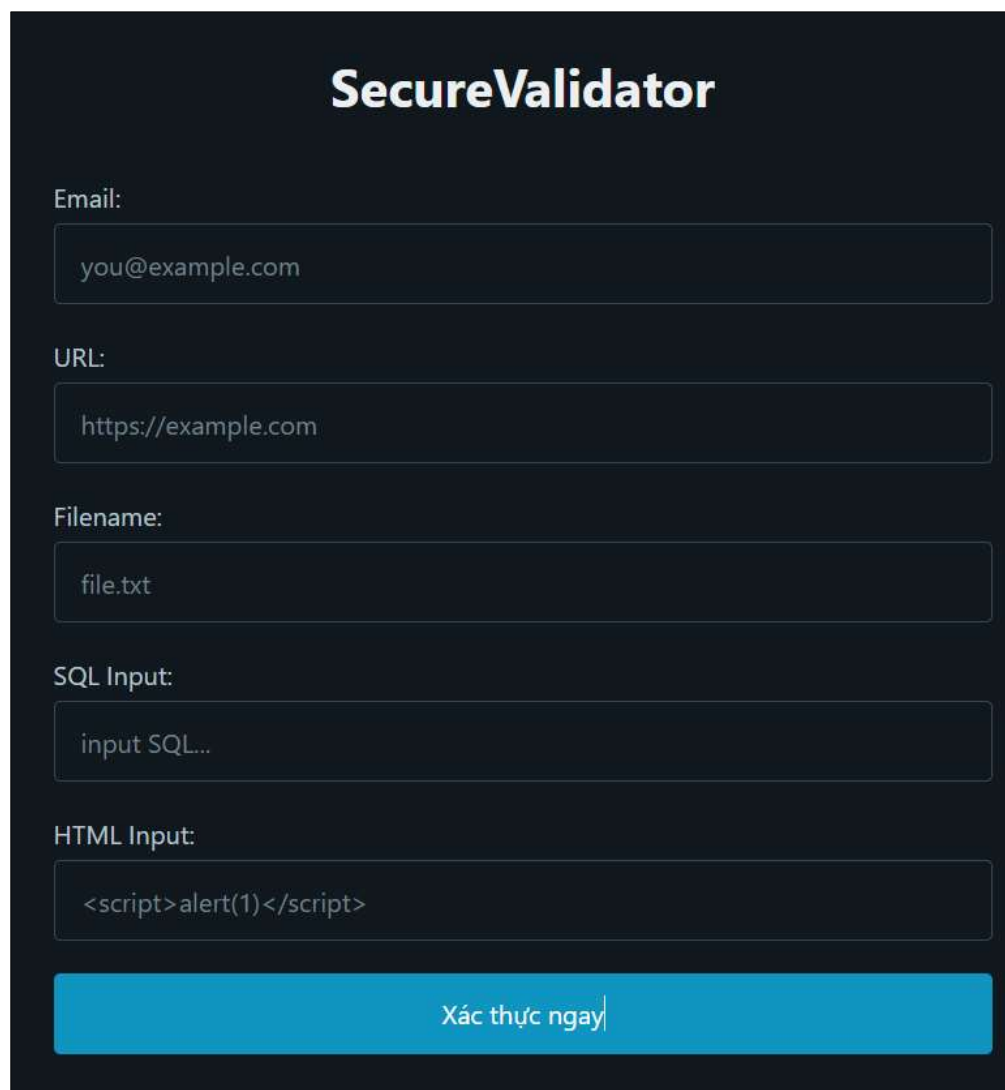
### 3. Kiểm tra trên giao diện:

- Cài các gói cần thiết: `pip install -r requirements.txt`
- Chạy file `app.py`:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\secure-validator-lab> python .\app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 993-054-488
```

- Mở trình duyệt > Vào `http://127.0.0.1:5000/` để kiểm tra.



The image shows a web application titled "SecureValidator" with a dark theme. It contains several input fields for testing: "Email:" with the value "you@example.com", "URL:" with "https://example.com", "Filename:" with "file.txt", "SQL Input:" with "input SQL...", and "HTML Input:" with "<script>alert(1)</script>". At the bottom, there is a large blue button labeled "Xác thực ngay" (Verify now).

- Kiểm tra các trường hợp (đúng/sai) và quan sát kết quả.

Email:

Email hợp lệ

URL:

URL hợp lệ

Filename:

Tên file không hợp lệ

SQL Input:

Đã lọc:

HTML Input:

Đã mã hóa:

#### 4. Deploy lên Render:

- Tạo file .gitignore:

```
.gitignore
1 *.pyc
2 __pycache__/.
3 .vscode/
4 .DS_Store
```

- Commit và push code lên Github:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git config --global user.email "hongphuoc2597@gmail.com"
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git config --global user.name "phuocntmh"
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git add .
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git commit -m "[add] secure validator lab"

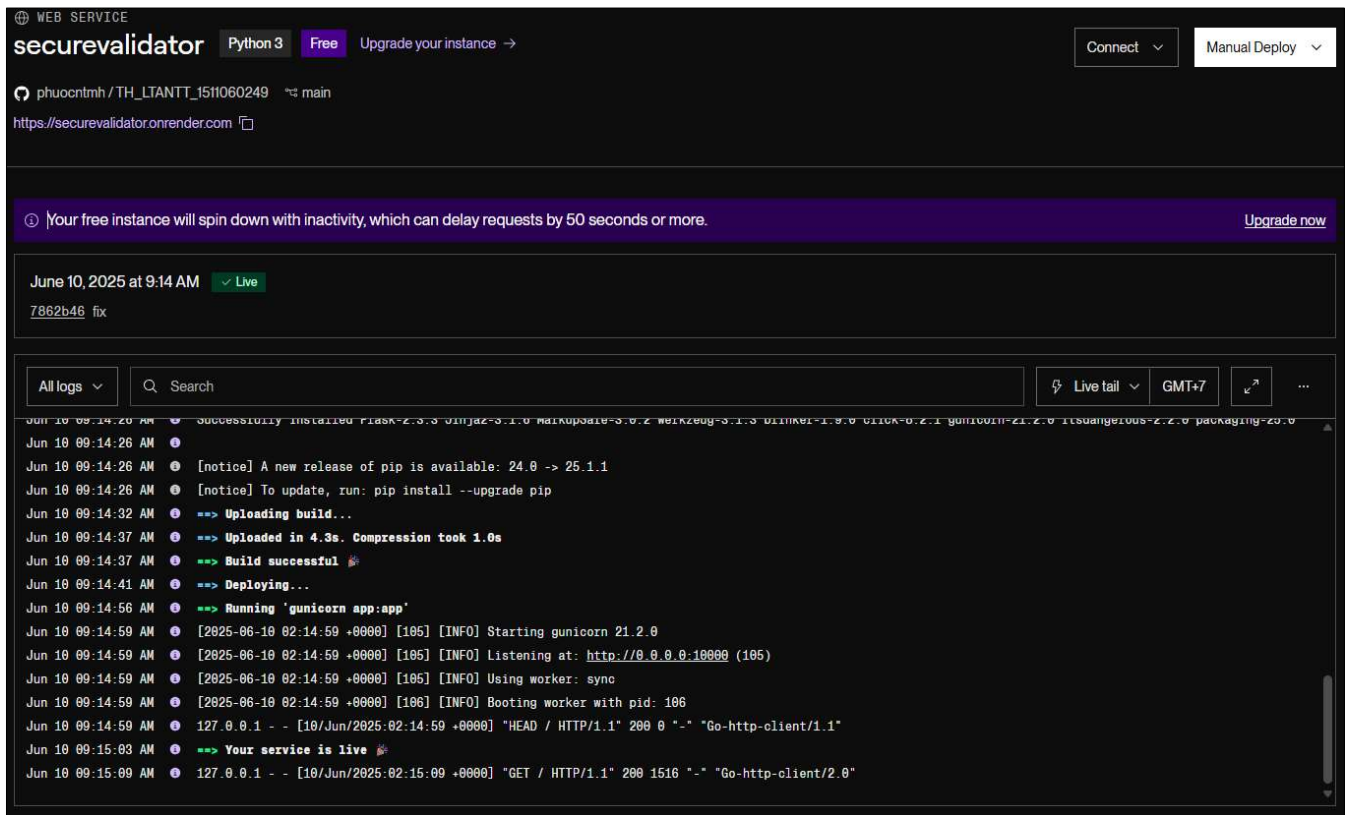
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git push origin main
Enumerating objects: 24, done.
Counting objects: 100% (24/24), done.
Delta compression using up to 8 threads
Compressing objects: 100% (19/19), done.
Writing objects: 100% (23/23), 6.63 KiB | 969.00 KiB/s, done.
Total 23 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/phuocntmh/TH_LTANTT_1511060249.git
   af03bd8..7862b46  main -> main
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> █
```

- Vào <https://dashboard.render.com/>
- Tạo tài khoản (đăng nhập bằng Github).
- Chọn "New + → Web Service". Kết nối với GitHub repo vừa push.
- Chọn branch chính (main). Thiết lập:

| Field         | Value                           |
|---------------|---------------------------------|
| Name          | securevalidator                 |
| Runtime       | Python                          |
| Root Document | secure-validator-lab            |
| Build Command | pip install -r requirements.txt |
| Start Command | gunicorn app:app                |
| Environment   | Python 3                        |

- app:app nghĩa là: file app.py và biến Flask tên app.
- Nhấn "Create Web Service". Render sẽ tự động:
  - Cài đặt môi trường > Build app > Chạy server.
  - Cung cấp một URL để test (dạng <https://your-app-name.onrender.com/>).
  - Ví dụ: <https://securevalidator.onrender.com/>



- Vào URL mà Render cung cấp, kiểm tra lại ứng dụng.

## 5. Cấu hình tự động bằng `render.yaml`:

- Tạo file `render.yaml` trong thư mục gốc. Nội dung:

```
secure-validator-lab > ! render.yaml
1  services:
2    - type: web
3      name: securevalidator
4      runtime: python
5      buildCommand: "pip install -r requirements.txt"
6      startCommand: "gunicorn app:app"
7      plan: free
8
```

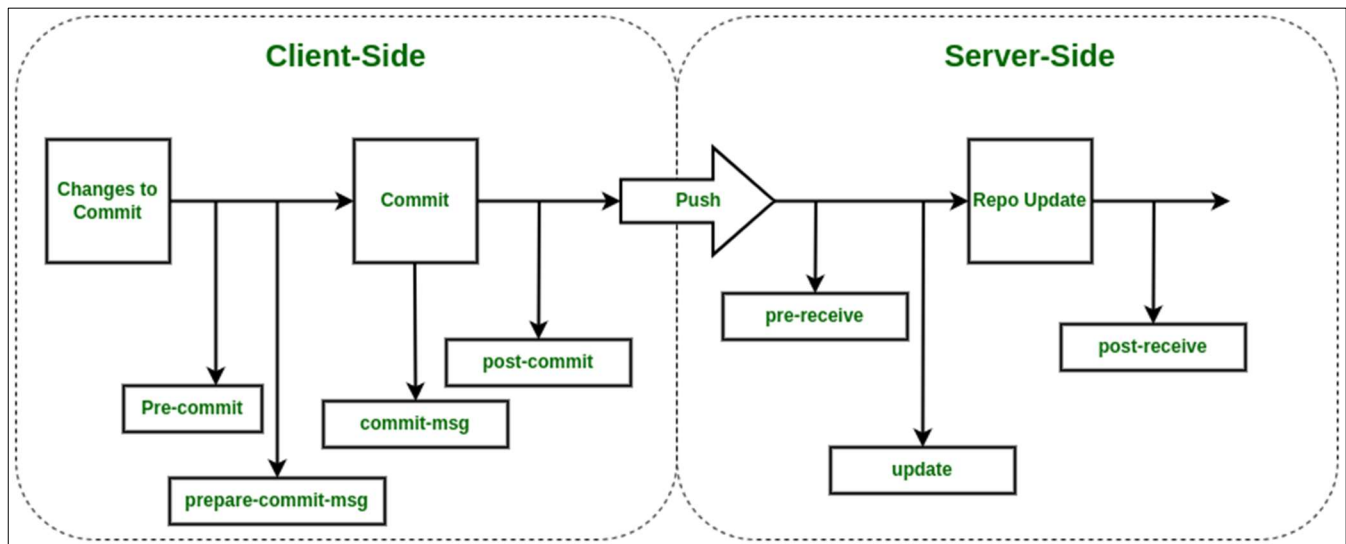
- Commit, push lại lên nhánh main. Tạo lại ứng dụng trên Render và kiểm tra.

## 1.3 GIT SECURITY VÀ PRE-COMMIT HOOKS

### 1.3.1 Sử dụng pre-commit hook để quét bảo mật

**Pre-commit hook** là script tự động chạy trước khi git commit, giúp phát hiện sớm lỗi bảo mật và ngăn mã không an toàn lọt vào repository. Lợi ích chính:

- Ngăn lỗi hỏng sớm: Phát hiện mã nguy hiểm, thông tin nhạy cảm ngay tại thời điểm commit.
- Tự động hóa kiểm tra: Không cần kiểm thủ công, tăng hiệu quả và độ tin cậy.
- Giáo dục bảo mật: Nhắc lập trình viên tuân thủ quy tắc an toàn khi viết và đẩy mã.



Hình 1.3. Git – Hooks. Nguồn: GeeksforGeeks

### 1.3.2 Phát hiện thông tin nhạy cảm trong mã nguồn

Rò rỉ thông tin nhạy cảm trong mã nguồn – như mật khẩu, API key, token truy cập là một trong những lỗi nghiêm trọng nhất, thường xảy ra do lập trình viên vô tình commit các dữ liệu quan trọng vào hệ thống quản lý mã nguồn. Rủi ro thường gặp:

- Lộ key truy cập hệ thống, dịch vụ đám mây, CSDL.
- Hacker khai thác từ các kho mã công khai.
- Mất an toàn, vi phạm chính sách bảo mật.
- Công cụ hỗ trợ phát hiện: GitLeaks, detect-secrets, TruffleHog, git-secrets: giúp quét mã và lịch sử git để phát hiện các chuỗi nhạy cảm.

### 1.3.3 Kiểm tra bảo mật tự động hóa

Kiểm tra bảo mật tự động hóa là quá trình sử dụng công cụ và quy trình tự động để phát hiện, phân tích các lỗ hổng và sai sót trong mã nguồn hoặc hệ thống, giảm thiểu lỗi do con người bỏ sót. Lợi ích:

- Tiết kiệm thời gian và công sức: Thay vì kiểm tra thủ công, công cụ tự động quét nhanh và liên tục.
- Phát hiện liên tục: Kiểm tra được thực hiện ở nhiều giai đoạn phát triển, từ commit đến build, deploy.
- Đảm bảo chất lượng bảo mật: Giúp duy trì tiêu chuẩn an toàn xuyên suốt vòng đời phần mềm.

## 1.4 THỰC HÀNH: BẢO MẬT TRƯỚC KHI COMMIT

---

### 1.4.1 Yêu cầu

#### Mục tiêu:

Thiết kế và triển khai một hệ thống pre-commit hook "GitSecure" có khả năng tự động kiểm tra mã nguồn trước khi thực hiện thao tác git commit, nhằm phát hiện và ngăn chặn các rủi ro bảo mật.

**Yêu cầu chức năng:** Hệ thống cần tích hợp các thành phần sau:

- Quét thông tin nhạy cảm: Phát hiện API keys, mật khẩu, token bị hardcode.
- Phát hiện thông tin định danh bị cài cứng: Nhận diện các thông tin xác thực cố định trong mã.
- Quét lỗ hổng cơ bản: Sử dụng công cụ Bandit để phân tích mã nguồn.
- Kiểm tra quyền truy cập file: Đảm bảo file không bị phân quyền sai, dễ bị khai thác.
- Kiểm tra tuân thủ giấy phép: Phát hiện các thư viện hoặc đoạn mã không phù hợp với quy định về bản quyền.

#### Yêu cầu kỹ thuật:

- Pre-commit hook phải tự động ngăn chặn việc commit nếu phát hiện bất kỳ vấn đề bảo mật nào.

- Hệ thống cần tạo một tệp log ghi lại chi tiết các phát hiện (findings) phục vụ việc xem xét và xử lý sau đó.

## 1.4.2 Hướng dẫn thực hành

- Tạo thư mục `.githubhooks`.
- Trong thư mục này tạo file `.githubhooks/pre-commit`:

```
.githubhooks > pre-commit > ...
1  #!/usr/bin/env python
2  import os, re, subprocess, sys, stat
3  from datetime import datetime
4
5  LOG_FILE = "gitsecure.log"
6  SENSITIVE_PATTERNS = [
7      r"apikey\s*=\s*['\"]([A-Za-z0-9_-]{16,})['\"]",
8      r"secret\s*=\s*['\"]([A-Za-z0-9_-]{8,})['\"]",
9      r"password\s*=\s*['\"]([^\"]{4,})['\"]",
10     r"token\s*=\s*['\"]([A-Za-z0-9]{10,})['\"]",
11     r"(AKIA|ASIA)[A-Z0-9]{16}"
12 ]
13
14 def log(msg):
15     with open(LOG_FILE, "a") as f:
16         f.write(f"[{datetime.now()}] {msg}\n")
17
18 def scan_sensitive(file_path):
19     try:
20         with open(file_path, "r", errors="ignore") as f:
21             content = f.read()
22             for pattern in SENSITIVE_PATTERNS:
23                 if re.search(pattern, content, re.IGNORECASE):
24                     return f"Sensitive info found in {file_path}: pattern {pattern}"
25     except Exception:
26         return None
27     return None
28
```

```

29 def check_permissions(file_path):
30     st = os.stat(file_path)
31     if st.st_mode & stat.S_IWOTH:
32         return f"File {file_path} is world-writable!"
33     return None
34
35 def run_bandit():
36     try:
37         result = subprocess.run(["bandit", "-r", "."], capture_output=True, text=True)
38         if "SEVERITY: High" in result.stdout:
39             log("Bandit: High severity issues found.")
40             return "Bandit: High severity issues found."
41     except FileNotFoundError:
42         return "Bandit not installed. Run: pip install bandit"
43     return None
44
45 def main():
46     findings = []
47     files = subprocess.check_output(["git", "diff", "--cached", "--name-only"]).decode().splitlines()
48     for file in files:
49         if not os.path.isfile(file): continue
50         result = scan_sensitive(file)
51         if result: findings.append(result)
52         perms = check_permissions(file)
53         if perms: findings.append(perms)
54     bandit_result = run_bandit()
55     if bandit_result: findings.append(bandit_result)
56

```

```

57     if findings:
58         print("\nCOMMIT BLOCKED by GitSecure:")
59         for f in findings:
60             print(" -", f)
61             log(f)
62         sys.exit(1)
63     else:
64         print("GitSecure: All checks passed.")
65
66 if __name__ == "__main__":
67     main()
68

```

- Đảm bảo script pre-commit có dòng "shebang" đúng đầu tiên.
- Gắn hook vào Git:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git config core.hooksPath .githooks
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> █

```

- Cấp quyền thực thi cho file pre-commit: Mở Git Bash tại folder dự án > Chạy lệnh:  
`chmod +x .githooks/pre-commit`

```
MINGW64:/c/Users/HONGPHUOC/Desktop/TH_LTANTT_1511060249
HONGPHUOC@PHUOCNTMH MINGW64 ~/Desktop/TH_LTANTT_1511060249 (main)
$ chmod +x .github/hooks/pre-commit
HONGPHUOC@PHUOCNTMH MINGW64 ~/Desktop/TH_LTANTT_1511060249 (main)
$
```

- Tạo file `requirements.txt` bên ngoài thư mục chính:

```
requirements.txt
1 bandit
```

- Cài các gói cần thiết: `pip install -r requirements.txt`
- Kiểm tra hook:
  - Tạo folder `pre-commit-hook-test`. Trong folder này tạo file `bad.py`:

```
pre-commit-hook-test > bad.py > ...
1 password = "123456"
```

- Commit thử:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git add .\pre-commit-hook-test\bad.py
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git commit -m "test"

COMMIT BLOCKED by GitSecure:
- Sensitive info found in pre-commit-hook-test/bad.py: pattern password\s*=\s*['\"][^'\"]{4,}['\"]
- File pre-commit-hook-test/bad.py is world-writable!
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249>
```

- Log được ghi vào `gitsecure.log`:

```
gitsecure.log
1 [2025-06-10 10:06:27.405874] Sensitive info found in pre-commit-hook-test/bad.py: pattern password\s*=\s*['\"][^'\"]{4,}['\"]
2 [2025-06-10 10:06:27.406336] File pre-commit-hook-test/bad.py is world-writable!
3
```

- Bỏ log khỏi repo: Cập nhật `.gitignore` > Thêm `gitsecure.log`.
- Xóa thông tin nhạy cảm > Commit lại > Push code lên Github.
- Nếu bị chặn vì quyền "world-writable" > Hãy chạy lệnh: `chmod 644 <tên_file>` trong Git Bash để thay đổi quyền ghi quá thoáng (777).
- Hoặc có thể cập nhật trong phương thức `check_permissions()` của hook:

```

29 def check_permissions(file_path):
30     if platform.system() == "Windows":
31         return False
32     st = os.stat(file_path)
33     if st.st_mode & stat.S_IWOTH:
34         return f"File {file_path} is world-writable!"
35     return None

```

- Kết quả cuối cùng:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git commit -m "[add] githooks"

COMMIT BLOCKED by GitSecure:
- File .githooks/pre-commit is world-writable!
- File .gitignore is world-writable!
- File pre-commit-hook-test/bad.py is world-writable!
- File requirements.txt is world-writable!
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git add .
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git commit -m "[add] githooks"
GitSecure: All checks passed.
[main 4d13d15] [add] githooks
4 files changed, 73 insertions(+), 1 deletion(-)
create mode 100644 .githooks/pre-commit
create mode 100644 pre-commit-hook-test/bad.py
create mode 100644 requirements.txt
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249>

```

## 1.5 LÀM SẠCH DỮ LIỆU, BẢO MẬT GHI NHẬT KÝ

### 1.5.1 Kỹ thuật làm sạch dữ liệu

**Làm sạch dữ liệu (Data Sanitization)** là quá trình loại bỏ hoặc vô hiệu hóa các thành phần nguy hiểm trong dữ liệu đầu vào nhằm ngăn chặn các tấn công như XSS, SQL Injection hay Command Injection. Nguyên tắc cơ bản:

- Không tin tưởng dữ liệu đầu vào.
- Làm sạch càng sớm càng tốt.
- Ưu tiên whitelist thay vì blacklist.
- Làm sạch theo ngữ cảnh sử dụng.

Kỹ thuật phổ biến:

- Escaping & Encoding: Vô hiệu hóa ký tự đặc biệt (<, >, ', "...).
- Lọc theo whitelist: Chỉ cho phép ký tự hợp lệ.
- Sử dụng thư viện chuyên dụng: bleach, DOMPurify...
- Chuẩn hóa dữ liệu: Kiểm tra định dạng đầu vào.

## 1.5.2 Xử lý thông tin định danh cá nhân

**Thông tin định danh cá nhân (Personally Identifiable Information - PII)** là các dữ liệu có thể dùng để nhận diện một cá nhân cụ thể, như họ tên, số CCCD, email, địa chỉ IP, hoặc thông tin tài chính. Nguy cơ nếu PII bị rò rỉ:

- Vi phạm pháp lý (theo luật bảo vệ dữ liệu như GDPR, PDP...).
- Gây mất uy tín hệ thống.
- Dễ bị lợi dụng cho các cuộc tấn công lừa đảo, chiếm đoạt danh tính.

Nguyên tắc xử lý PII:

- Thu thập tối thiểu: Chỉ lưu trữ thông tin thực sự cần thiết.
- Ẩn hoặc mã hóa: Sử dụng kỹ thuật như hashing, encryption, masking.
- Kiểm soát truy cập: Giới hạn quyền truy xuất thông tin PII.
- Ghi log có chọn lọc: Tuyệt đối không ghi thô dữ liệu nhạy cảm vào log.

## 1.5.3 Phòng chống chèn mã vào nhật ký

**Chèn mã vào nhật ký (Log Injection)** là kỹ thuật tấn công khi kẻ xấu lợi dụng việc ghi log không an toàn để chèn các đoạn mã độc hoặc lệnh đặc biệt, gây ảnh hưởng đến hệ thống phân tích hoặc tạo lỗi. Rủi ro:

- Làm sai lệch dữ liệu log, khó phát hiện sự cố thật.
- Gây lỗi hoặc khai thác lỗ hổng qua công cụ phân tích log.
- Tiết lộ thông tin nhạy cảm hoặc gây tấn công chuỗi lệnh.

Nguyên tắc phòng chống:

- Lọc và mã hóa dữ liệu đầu vào trước khi ghi log, loại bỏ ký tự đặc biệt (newline, tab, escape sequence).
- Dùng định dạng log cố định, không cho phép chèn động nội dung nguy hiểm.
- Giới hạn quyền truy cập log, bảo vệ tính toàn vẹn dữ liệu.
- Kiểm tra và giám sát log định kỳ để phát hiện dấu hiệu bất thường.

## 1.6 THỰC HÀNH: GHI NHẬT KÝ ƯU TIÊN BẢO MẬT

### 1.6.1 Yêu cầu

**Mục tiêu:** Xây dựng hệ thống ghi nhật ký (SecureLogger) với các tính năng sau:

- Hỗ trợ đa cấp độ log (DEBUG, INFO, WARNING, ERROR, CRITICAL).
- Tự động phát hiện và che dấu thông tin định danh cá nhân (PII).
- Quản lý luân phiên log (log rotation) kèm nén dữ liệu.
- Phát hiện thay đổi trái phép trên các tập tin nhật ký (tamper detection).
- Ghi nhật ký theo cấu trúc JSON.

**Yêu cầu:** Tích hợp với thư viện SecureValidator từ bài tập trước để ghi lại tất cả các lần kiểm tra validation.

### 1.6.2 Hướng dẫn thực hành

- Tạo thư mục `secure_logger_lab`. Trong thư mục này, tạo các thư mục và tệp tin theo cấu trúc sau:

```
secure-validator-lab/  
├─ app.py  
├─ securevalidator/  
├─ securelogger/  
|   ├─ __init__.py  
|   └─ logger.py  
└─ requirements.txt
```

- Sao chép các file từ folder “secure-validator-lab/securevalidator” sang folder “secure\_logger\_lab/ securevalidator”.
- Viết module `logger.py`:

secure\_logger\_lab > securelogger > logger.py > ...

```
1 import logging, logging.handlers, gzip, json, os, re, hashlib
2 from datetime import datetime
3
4 LOG_FILE = "secure.log"
5 SIGNATURE_FILE = "secure.log.sig"
6 MAX_LOG_SIZE = 1024 * 1024
7 BACKUP_COUNT = 2
8
9 PII_PATTERNS = {
10     "email": r'[\w\.-]+@[ \w\.-]+\.\w+',
11     "token": r'(?i)(token|apikey|key|password)\s*=\s*["\']?[\w\.-]{8,}["\']?',
12 }
13
14 def mask_pii(text):
15     for label, pattern in PII_PATTERNS.items():
16         text = re.sub(pattern, f"<{label}>_masked>", text, flags=re.IGNORECASE)
17     return text
18
19 def hash_line(line):
20     return hashlib.sha256(line.encode('utf-8')).hexdigest()
21
22 def append_signature(line):
23     with open(SIGNATURE_FILE, "a", encoding="utf-8") as f:
24         f.write(hash_line(line))
25
```

```
26 class JSONFormatter(logging.Formatter):
27     def format(self, record):
28         message = mask_pii(record.getMessage())
29         record_dict = {
30             "timestamp": datetime.utcnow().isoformat() + "Z",
31             "level": record.levelname,
32             "message": message,
33         }
34         if hasattr(record, "data"):
35             record_dict["data"] = mask_pii(str(record.data))
36         if hasattr(record, "results"):
37             record_dict["results"] = mask_pii(str(record.results))
38         json_line = json.dumps(record_dict, ensure_ascii=False)
39         return json_line
40
41 class GZipRotator:
42     def __call__(self, source, dest):
43         with open(source, 'rb') as f_in, gzip.open(dest + ".gz", 'wb') as f_out:
44             f_out.writelines(f_in)
45         os.remove(source)
46
```

```

47 class SecureRotatingFileHandler(logging.handlers.RotatingFileHandler):
48     def emit(self, record):
49         try:
50             msg = self.format(record)
51             super().emit(record)
52             append_signature(msg)
53         except Exception:
54             self.handleError(record)
55
56     def get_secure_logger():
57         logger = logging.getLogger("secure_logger")
58         logger.setLevel(logging.DEBUG)
59         if not logger.handlers:
60             handler = SecureRotatingFileHandler(
61                 LOG_FILE, maxBytes=MAX_LOG_SIZE, backupCount=BACKUP_COUNT, encoding="utf-8"
62             )
63             handler.setFormatter(JSONFormatter())
64             handler.rotator = GZipRotator()
65             logger.addHandler(handler)
66         logger.propagate = False
67         return logger
68
69 secure_logger = get_secure_logger()
70

```

- Code cho file securelogger/ \_\_init\_\_.py:

secure\_logger\_lab > securelogger >  \_\_init\_\_.py

```

1 from securelogger.logger import get_secure_logger

```

- Code cho file app.py:

secure\_logger\_lab >  app.py > ...

```

1 from flask import Flask, request, jsonify
2 from securevalidator.core import (
3     validate_email, validate_url, validate_filename,
4     sanitize_sql_input, sanitize_html_input
5 )
6 from securelogger.logger import get_secure_logger
7
8 app = Flask(__name__)
9 secure_logger = get_secure_logger()
10

```

```

11 @app.route("/validate", methods=["POST"])
12 def validate():
13     try:
14         data = request.get_json(force=True)
15     except Exception as e:
16         secure_logger.warning("Invalid JSON received",
17                               extra={"data": str(request.data)})
18         return jsonify({"error": "Invalid JSON format"}), 400
19
20     results = {
21         "email": validate_email(data.get("email", "")),
22         "url": validate_url(data.get("url", "")),
23         "filename": validate_filename(data.get("filename", "")),
24         "sql": sanitize_sql_input(data.get("sql", "")),
25         "html": sanitize_html_input(data.get("html", "")),
26     }
27
28     secure_logger.info("Validation check performed",
29                       extra={"data": data, "results": results})
30     return jsonify(results)
31
32 if __name__ == "__main__":
33     app.run(debug=True)
34

```

- File requirements.txt:

secure\_logger\_lab > requirements.txt

1 Flask

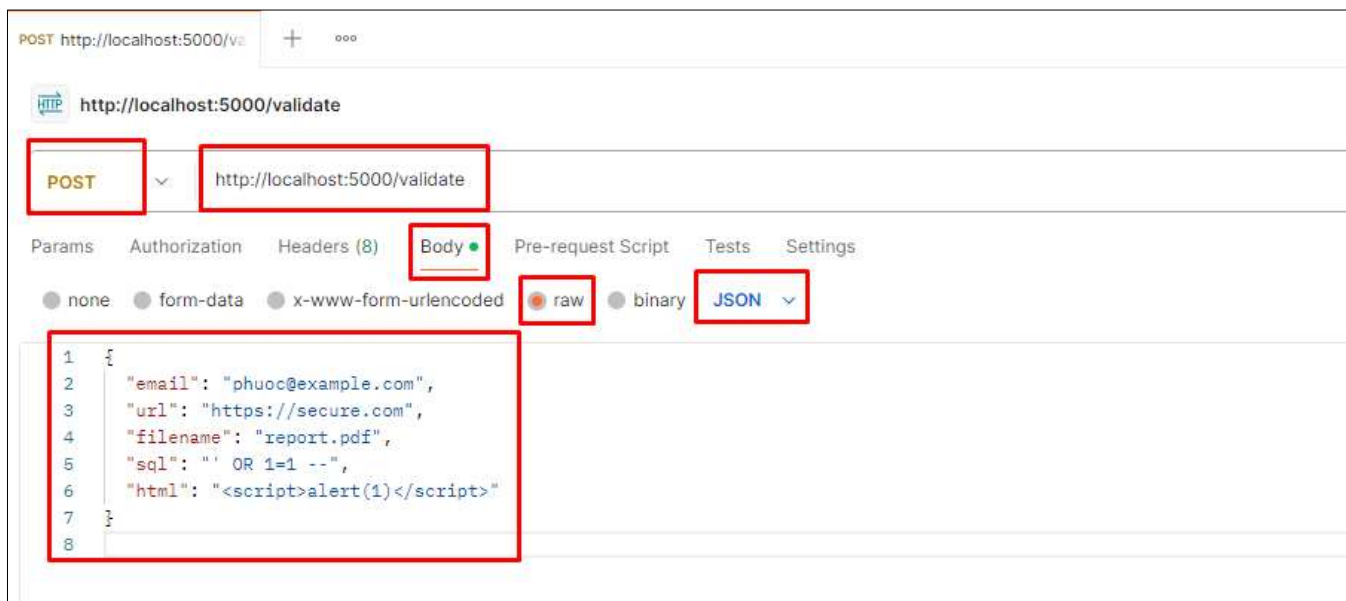
- Cài các gói cần thiết: `pip install -r requirements.txt`
- Chạy ứng dụng: `python app.py`

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> cd .\secure_logger_lab\
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\secure_logger_lab> python .\app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 993-054-488

```

- Tải và cài đặt Postman tại: <https://www.postman.com/downloads/>
- Sau khi cài xong, mở Postman lên, tạo mới 1 request như sau:



- Chọn nút "Send" để kiểm tra. Ta thu được kết quả:



- Kiểm tra file `secure.log`> Ta thấy đã có ghi nhận log, phần PII đã bị che.



- Kiểm tra file `secure.log.sig`> Ta thấy có kết quả băm của phần log đã được ghi nhận. Phần này dùng để đi xác thực dữ liệu log có bị thay đổi hay không.
- Sau khi thực hiện xong, tiến hành commit, push code lên Github.